

The error message was telling the truth and lying at the same time

Slice books-27, part one — August 21, 2026

Michael, you asked me to fix the error you hit running migration 0195, and you asked me not to guess. So let me give you the short version first, then show my work.

The file is fine. Nothing in it is broken. The error was real, but the cause was not in the file — it was in how the file got to the database.

I know that is an unsatisfying sentence, so I am going to earn it.

What the error actually said

You saw this:

```
Failed to run sql query: ERROR: 42P01: relation "a" does not exist
```

In plain English, Postgres is saying: *you asked me for a table called "a", and there is no table called "a"*. And the natural next thought — the one I had, and the one anybody would have — is that somebody misspelled a table name.

Nobody did. There is no table named "a" in your system and there was never meant to be. The word "a" got in front of the database's eyes some other way.

How I proved the file is good

I did not read the file and form an opinion. I installed a real PostgreSQL database in this workspace, set it up the way Supabase sets yours up, and ran the migrations at it. Three separate ways:

First, I applied all 195 migrations in order, from an empty database. Clean. No errors.

Second, I applied 0195 a *second* time on top of itself, because you run these by hand and a hand can slip. Clean again — re-running it is safe.

Third, and this is the one that matters most, I rebuilt a database in exactly the state yours was in when it failed — migrations 0001 through 0194 applied, and 0195 not yet. Then I ran 0195. It applied cleanly, the built-in audit came back with zero problems, and all four tables it is supposed to create were there: `employee_w4`, `employee_i9`, `employee_pay`, `employee_ssn_reveals`.

The file works. On your database's exact starting state.

So where does the word "a" come from?

I searched 0195 for anything that looks like asking for a table named "a". There is exactly one line in the whole file, line 490, and it is a **comment** — a note to a human being, not an instruction to the computer:

```
-- the service role during onboarding, not edited from a screen.
```

Read the tail of that sentence on its own: *"from a screen."* Now imagine the two dashes at the front going missing. `--` is how SQL says *"ignore the rest of this line, it is a note."* Take those two characters away and the database no longer sees a note. It sees an instruction. It reads "from a" and goes looking for a table named "a", does not find one, and reports exactly the error you saw.

Eight other comment lines in that file also end in a bare "a". Any one of them could do the same thing.

And here is the detail that makes this more than a theory: that line sits inside a block of code wrapped in special quoting marks. Some tools try to be clever and strip comments out before sending SQL along. A tool that does not understand that particular kind of quoting will make a mess precisely there.

What I could not prove, and am not going to pretend I did

I could not reproduce it. I tried hard. I chopped the file up the naive way a simple tool would (77 pieces — plenty of errors, never yours). I tried several comment-stripping approaches. I tried word-wrapping the file at six different widths, the way an editor might reflow a long line. I tried every one of the 716 possible partial pastes of the end of the file. I tried cutting it off every 500 characters, from character 1,000 all the way to the end at 38,047.

None of them produced your error. The only thing that produces your exact message is literally handing the database the words `select * from a screen`.

So I know **what** happened — prose reached the parser. I do not know **which piece of software** did it, and I am not going to name a culprit I cannot prove. You asked me never to guess.

What to do if it happens again: re-send the whole file, in one paste, from top to bottom. Do not paste it in pieces, and do not let anything reformat it on the way. If it still fails, tell me and I will chase the client itself.

The part that bothers me more than the bug

Migration 0195 has **545 lines of tests**. Every one of them passing. Green.

Not one of them could have caught this, and I want to be honest with you about why, because it is the kind of thing that looks fine on a status report and is not fine.

Every one of those tests reads the migration **as a piece of text**. They open the file and check that it *says* the right things — that a rule is present, that a column is spelled a certain way. That is genuinely useful. It catches the day somebody quietly deletes a safety check.

But reading a recipe is not cooking. Not one test in this entire repository had ever handed a migration to an actual database and asked *does this run*? We had 195 migrations, thousands of tests, and zero proof that any of them would execute.

You found that gap by hitting it.

What I built so it cannot happen again

A check that runs the SQL instead of reading it. It builds a database, applies all 195 migrations in order, and stops at the first one that fails. Then it applies the last one a second time, to prove re-running is safe — because you apply these by hand.

And it explains itself in English. When it sees `relation "a" does not exist`, it notices that "a" is an ordinary English word rather than something anyone would name a table, and instead of leaving you hunting for a missing table it prints:

The database was asked for a table named "a". That is an ordinary English word, not a table anyone would create, which means the server was almost certainly sent PROSE instead of SQL — a comment line whose "--" marker went missing between the file and the database. The migration file itself is probably fine. Suspect the transport. Re-send the WHOLE file.

That is the message you should have gotten in the first place.

And it now runs automatically. Every change from here on gets all 195 migrations executed against a real database before it can merge. It ran on this change: 195 migrations applied in 13 seconds, 0195 re-applied cleanly, passed.

I also checked the check, in both directions — a gate that cannot fail is not a gate. I deliberately broke 0195 by putting `select * from a screen;` into it, and the new check caught it and printed the explanation above. I removed the `--` from line 490, and it caught that too. Then I put the file back exactly as it was and confirmed it byte for byte.

While I was at it I scanned all 195 migrations for any *genuine* reference to a table named "a" — not comments, not text inside quotes, the real thing. **Zero.**

Your filed returns: you were right, Sage is wrong

You said *"sage is probably wrong, our research is recent... so our system has the right data i think."*

You were right. And two of the four problems I reported to you last time were **my mistakes**, not Sage's. I want to correct them plainly.

The reason I got them wrong is worth stating: I had been reading Sage's **printouts**. You then sent me the returns **as actually filed with the agencies**. Those are not the same kind of document. A printout says what a program believes. A filed return says what the government received and charged you. When they disagree, the filed return wins, every time.

Your unemployment rate — you were right. The filed return prints the rates itself: UI **0.37%** and EAF **0.03%**, on wages of 68,923.45. That gives 255.02 and 20.68, totalling the 275.70 the State actually assessed. Sage's 0.64% would have produced 441.11 — overstating your UI by **\$186.09 for the quarter**.

Nothing was overpaid. The State billed from its own rate, so your money is fine. But **Sage's rates need fixing for the rest of 2026**, and that is the one item on this page that needs your hands:

in Sage	change from	change to
Unemployment (UI)	0.64%	0.37%
Employment Admin Fund (EAF)	0.64%	0.03%

Paid Family & Medical Leave — I was wrong, withdraw that one. I told you the small-employer exemption might not be getting claimed, and that there might be around \$890 a year to recover. The filed return shows Employer Medical **0.00** and Employer Family **0.00**. The exemption is being taken, correctly. There is no money to recover because there was never an overpayment. That figure I flagged came off a Sage worksheet, not off the return.

WA Cares — I was wrong about that too. I told you 399.76 had been withheld but reported as zero. The filed return reports **399.76**, to the cent. The zeros were an artefact of the printout. Nothing is missing from the State's records.

L&I — confirmed. One risk class, 6403-05, Stores: Specialty Groceries. 3,558 hours × 0.5593 = **1,989.99** exactly.

Your 941 — and a genuinely good sign. I recomputed every line and they all match. Line 12, total tax for the quarter: **\$14,204.57**.

Two things about that number.

First, your wage figure of **68,923.45** is *identical* on all three filings — the unemployment return, the L&I report, and the 941. Three agencies, three separate filings, one wage base, no drift. That is exactly the cross-check that makes this quarter solid ground to build on.

Second — and I did not arrange this — **\$14,204.57 is precisely the number the deposit-schedule engine was built on last slice**, before you sent me the return. It was computed from your data and the filed return has now confirmed it independently. On top of that, your 941 came with a **Schedule B** attached, and only semiweekly depositors file Schedule B. So the conclusion that you are on the semiweekly schedule is now confirmed twice: once by our arithmetic, once by the government's own paperwork.

Where that leaves the four problems I reported:

#	what I told you	where it stands now
1	EAF charged at 0.64% instead of 0.03%	Real. Fix in Sage.
2	UI rate conflict, 0.37% vs 0.64%	Real — 0.37% is right. Fix in Sage.
3	PFML exemption possibly unclaimed	Withdrawn. My error.
4	WA Cares reported as zero	Withdrawn. My error.

Two real, two mine. I would rather tell you that than quietly leave four on the list.

The SSNs are gone

You asked me to leave the raw text of the deleted PDFs out entirely. Done — I deleted the whole extraction folder. It contained four full Social Security numbers. None of it was ever committed to the repository. The *conclusions* live on in the permanent record; the raw personal data does not.

Your question about Form 940

You asked whether the 940 comes in this slice or a future one. **It is the next slice, books-28.**

Here is the running order, so it is in front of you:

- **books-27** (this one) — the reports engine, and reconciling your 941s against your W-2s
- **books-28** — the federal forms: **Form 940**, the 941 itself, W-2 and W-3, and 1099-NEC

- **books-29** — the Washington forms
- **books-30** — filing and distribution

One thing worth knowing now because it shapes books-28: since you are a semiweekly depositor, your 941 is incomplete without Schedule B, and Schedule B reports what you owe **day by day** — not as a quarterly total. That means the daily figure has to be captured as payroll runs, not reconstructed at filing time. The system is now set up to do that.

And to be explicit about the boundary, because it affects what you will still need elsewhere: we are replacing the **data preparation** half of what Aatrix does for you. We are not becoming a filing agent. The system will produce correct, complete, checked figures and the forms that carry them. Transmitting them to the agencies stays with you.

Where this leaves us

The migration you could not run is proven sound, and the reason nobody caught the problem — that we had never once executed a migration in testing — is now closed with a check that runs every time.

Your instinct about Sage was right, and it cost me two false alarms to learn that a printout and a filed return are different kinds of evidence. I have written that down as a standing rule so I do not repeat it.

Everything checked: 369 test files, 7,665 tests, all passing. Type checking clean. All 195 migrations executed for real, in the automated pipeline, on this change.

The one thing waiting on you is the two Sage rates: UI to **0.37%**, EAF to **0.03%**.

Nothing here was guessed. Where I could not prove something — which piece of software mangled your file on the way to the database — I have said so plainly rather than hand you a confident answer I could not stand behind.